

Prueba end-to-end: WhatsApp automático con Evolution API

Objetivo: verificar que las **notificaciones automáticas por WhatsApp** ya implementadas en STApp funcionan de punta a punta usando **Evolution API** (provider no oficial, vía QR). Esta guía NO implementa nada nuevo — prueba lo que ya existe en el código.

Resultado esperado al terminar: al **cambiar el estado de una orden**, el cliente recibe un WhatsApp automático con la plantilla del nuevo estado.

Qué ya funciona (y qué no)

Evento	¿Notifica automático hoy?
Orden cambia de estado (RECIBIDO → PRESUPUESTADO → ...)	✅ Sí (<code>ordenes/[id]/route.ts:374-402</code>)
Presupuesto definido	✅ Sí
Orden entregada	✅ Sí
Recordatorio de retiro (cron)	⚠ Solo email, no WhatsApp
Orden creada / cargada	❌ No — trigger ausente (gap conocido)

Esta prueba cubre el caso ✅ (cambio de estado). El caso de "orden cargada → WhatsApp" requiere código nuevo y NO se prueba acá.

Prerequisitos

1. **Servidor Evolution API corriendo y accesible por HTTPS**. Es un servicio aparte que self-hosteás (no viene con STApp). Repo: <https://github.com/evolution-foundation/evolution-api>
2. Un **número de WhatsApp** dedicado para el taller (el que se va a vincular por QR). Idealmente NO tu WhatsApp personal — Evolution es no oficial y existe riesgo de baneo.
3. Un **número de cliente de prueba** distinto, donde recibir los mensajes.
4. Cuenta STApp con **plan Profesional** (o un plan con el feature `whatsapp_notifications` activo). Sin esto, la config de WhatsApp devuelve 403.
5. Acceso **ADMIN** a la org de prueba.

Parte A — Levantar el servidor Evolution API

Si ya tenés un Evolution corriendo, saltá a la Parte B con su `baseUrl` y `apiKey`.

Forma más rápida (Docker, en un VPS con dominio + HTTPS):

```
docker run -d \
  --name evolution-api \
  -p 8080:8080 \
  -e AUTHENTICATION_API_KEY="GENERA_UNA_CLAVE_LARGA_Y_SECRETA" \
  -e DEL_INSTANCE=false \
  atendai/evolution-api:latest
```

Anotá:

- **baseUrl**: la URL pública del server, ej. <https://evo.tudominio.com> (debe ser HTTPS y resoluble desde donde corre STApp).
- **apiKey**: el valor de `AUTHENTICATION_API_KEY`.
- **instanceName**: un nombre que vas a elegir vos en la Parte B (ej. `taller-centro`). Se crea solo al guardar la config.

Importante: el server debe ser alcanzable desde el backend de STApp (Vercel). Un Evolution en `localhost` NO sirve para el deploy en producción.

Parte B — Conectar Evolution en STApp (QR pairing)

1. Entrá a STApp con la org de prueba → **Configuración** → **WhatsApp**.
2. Elegí la pestaña **Evolution**.
3. Cargá:
 - **Base URL**: la `baseUrl` de la Parte A.
 - **Instance name**: ej. `taller-centro`.
 - **API key**: la `apiKey` de la Parte A.
4. Guardá. STApp crea la instancia en el server (idempotente) y consulta su estado.
5. Tocá **Solicitar QR**. Aparece un QR (y/o pairing code).
6. En el teléfono del taller: WhatsApp → **Dispositivos vinculados** → **Vincular un dispositivo** → escaneá el QR.
7. El estado debe pasar a `open` (conectado). STApp marca la instancia como verificada.

✅ Checkpoint: estado de conexión = `open`.

Parte C — Activar las notificaciones por WhatsApp

El canal WhatsApp tiene un interruptor por organización, **separado** de la conexión.

1. **Configuración** → **Notificaciones**.
2. Activá **Notificaciones por WhatsApp**.
3. Guardá.

Bajo el capó esto setea el flag de la org que `lib/notifications/send-direct.ts:261` chequea antes de enviar. Sin este toggle en ON, el cambio de estado NO dispara WhatsApp aunque Evolution esté conectado.

Parte D — Smoke de envío manual (aísla la conexión)

Antes de probar el flujo automático, confirmá que el canal envía:

1. **Configuración** → **WhatsApp** → **botón de prueba** (envía un mensaje fijo a un número que ingresás).
2. Ingresá el **número del cliente de prueba** (formato local está bien: STApp normaliza a 54... y maneja el 15).
3. Enviá.

✅ Checkpoint: Llega el mensaje de prueba al cliente. ❌ Si NO llega: el problema es la conexión/credenciales Evolution, no el flujo de órdenes. Resolvé esto antes de seguir (ver Troubleshooting).

Parte E — Prueba del flujo AUTOMÁTICO (el objetivo real)

1. Asegurate de tener un **cliente de prueba** con el **telefono** cargado (el número donde vas a recibir).
2. Creá una **orden** para ese cliente (o usá una existente).
 - Nota: la creación en sí NO manda WhatsApp (gap conocido).
3. Abrí la orden y **cambiá su estado** (ej. RECIBIDO → PRESUPUESTADO, o → EN_REPARACION).
4. Guardá la transición.

✅ Checkpoint principal: a los segundos, el **cliente recibe un WhatsApp** con la plantilla del nuevo estado (ej. "Tu equipo pasó a estado: Presupuestado...").

5. Repetí con otro estado (ej. → REPARADO) y verificá que llega el mensaje correspondiente.
 6. Probá **definir/cambiar el presupuesto** → debe llegar la notificación de presupuesto.
 7. **Entregá** la orden (botón entregar) → debe llegar la notificación de entrega.
-

Parte F — Verificación en base de datos (evidencia)

Corré en el SQL editor (reemplazá el **organization_id** de tu org de prueba):

```
-- Mensajes de WhatsApp registrados (estado: sent/delivered/read/failed)
SELECT whatsapp_message_id, phone_number, template_name, status,
       error_code, error_message, created_at
FROM whatsapp_messages
WHERE organization_id = 'TU_ORG_ID'
ORDER BY created_at DESC
LIMIT 20;

-- Log de notificaciones (intentos por canal)
SELECT tipo, canal, status, message_id, created_at
FROM notification_logs
WHERE organization_id = 'TU_ORG_ID'
ORDER BY created_at DESC
LIMIT 20;
```

✅ Esperado: filas recientes con **status** = sent (o delivered) y **canal** = WHATSAPP para cada cambio de estado que hiciste.

Troubleshooting — si NO llega el WhatsApp automático

Revisá en orden (de más común a menos):

1. **Plan:** ¿la org tiene `whatsapp_notifications`? Sin el plan Profesional, la config devuelve 403. Verificá el plan en Superadmin.
2. **Toggle de canal:** ¿Configuración → Notificaciones → WhatsApp está en ON? Es el gate de `send-direct.ts:261`.

```
-- Verificar el flag de WhatsApp y el teléfono del cliente
SELECT id, nombre, slug FROM organizations WHERE id = 'TU_ORG_ID';
SELECT id, nombre, telefono FROM clientes WHERE id = 'CLIENTE_ID';
```

3. **Conexión Evolution:** estado debe ser `open`.

```
SELECT provider, evolution_base_url, evolution_instance_name,
       evolution_connection_state, is_verified
FROM whatsapp_config WHERE organization_id = 'TU_ORG_ID';
```

Si `evolution_connection_state <> 'open'` → re-escaneá el QR (Parte B). El pairing se cae si el teléfono pierde sesión.

4. **Teléfono del cliente:** la orden notifica al `clientes.telefono`. Si está vacío o mal cargado, no hay a dónde enviar. STApp normaliza a `54...`, pero un número inválido falla silencioso.
5. **Server alcanzable:** ¿el `baseUrl` de Evolution responde por HTTPS desde internet (no localhost)? El backend de STApp (Vercel) tiene que poder llegar.
6. **Errores en `whatsapp_messages`:** si hay filas con `status = failed`, mirá `error_code` / `error_message` — ahí está la causa real (número inválido, instancia desconectada, rate limit del server Evolution).
7. **El envío es fire-and-forget:** si falla, hoy NO hay reintento automático — solo queda el log. Un fallo transitorio del server Evolution se pierde.

Pista de diseño: el envío automático lee el flag de notificaciones WhatsApp de la org en `send-direct.ts`, mientras que el toggle de UI lo escribe vía `/api/notificaciones/config`. Si activaste el toggle y aún así no envía, confirmá en DB que el flag quedó persistido (punto 2) antes de buscar más lejos.

Resumen

- Evolution + envío automático por **cambio de estado** YA está en el código; esta guía solo lo configura y verifica.
- Los 3 gates que tienen que estar TODOS en verde: **plan** (`whatsapp_notifications`) + **toggle de canal** (Notificaciones → WhatsApp) + **conexión Evolution** (`open`).
- "Orden creada → WhatsApp" y "recordatorio por WhatsApp" son gaps reales: requieren código nuevo, fuera de esta prueba.